

Parcial II (Examen Práctico): Evaluación de rendimiento del algoritmo NEAT en clasificación de polaridad de textos

Brian Miguel Escalona Maldonado (21-003-1605)

brian.escalona@alumnos.uacm.edu.mx

Resumen—En este trabajo se evalúa el rendimiento del algoritmo evolutivo NEAT (NeuroEvolution of Augmenting Topologies) en la clasificación multiclase de polaridad de textos (positivo, negativo y neutro) utilizando un corpus de tweets en español. Se exploran dos tipos de representaciones vectoriales: TF-IDF (con reducciones de dimensionalidad al 20 %, 25 % y 30 % mediante TruncatedSVD) y Word Embeddings densos regionales de FastText (reducidos mediante PCA). La experimentación se estructuró en tres fases de análisis de sensibilidad para examinar el impacto de la representación de datos, la dinámica estructural y los componentes de diversidad y especiación del algoritmo. Los resultados revelan que la mejor configuración de NEAT alcanzó un F1-score macro de 0.4909 empleando Word Embeddings al 30 % de dimensionalidad y un alto umbral de compatibilidad.

I. INTRODUCCIÓN

En este examen práctico, evaluaremos el algoritmo NEAT (NeuroEvolution of Augmenting Topologies) como método evolutivo para la evolución de redes neuronales aplicadas a un problema de clasificación de textos.

I-A. Conjunto de datos

El conjunto de datos para la experimentación es un conjunto de tweets de polaridad clasificados en tres clases:

1. Positivo
2. Negativo
3. Neutro

Cada línea del archivo representa un **json**, que consta de los campos:

- **id**: Define el identificador del texto.
- **klass**: Define la clase del texto (positive, negative, neutral).
- **text**: Contiene el texto (tweet).
- **we_ft**: Contiene los vectores densos (embeddings) de 300 dimensiones creados con **FasText** para el español general.
- **we_mx**: Contiene los vectores densos (embeddings) de 300 dimensiones creados con **FasText** para el español de México.
- **we_es**: Contiene los vectores densos (embeddings) de 300 dimensiones creados con **FasText** para el español de España.

I-B. Representación de datos

Se utilizarán dos representaciones vectoriales diferentes de los datos:

- **TF-IDF**: Una representación dispersa basada en la frecuencia de términos. Se probarán diferentes porcentajes de características (20 %, 25 % y 30 %).
- **WordEmbeddings**: Vectores densos preentrenados de **FastText** de 300 dimensiones y tres regiones diferentes.

Con ayuda de NEAT, evolucionaremos de manera simultánea los pesos y la topología de nuestras redes neuronales. Estas redes serán evaluadas en un conjunto de prueba y comparadas con clasificadores **SVM** tradicionales y optimizados, utilizando los mismos preprocesamientos y representaciones vectoriales.

I-C. Conjunto de datos

Para el entrenamiento y la experimentación, se utilizarán dos conjuntos de datos con tweets de polaridad clasificados en tres clases (positivo, negativo, neutro). El conjunto de entrenamiento incluye 3,142 ejemplos de entrenamiento y 786 datos de experimentación.

II. DESARROLLO

Para realizar la experimentación de parámetros del algoritmo NEAT, se definieron los parámetros a probar en la *Tabla*: I. Para evitar una explosión de combinaciones, se realizará una experimentación por fases; esto servirá para realizar un análisis de sensibilidad, probando de manera aislada distintos parámetros para evaluar como afecta el rendimiento del algoritmo. La experimentación en esta ocasión se llevará a cabo en tres fases, en las cuales probaremos distintos parámetros; las redes generadas por el algoritmo después de cada experimento serán evaluadas contra distintos clasificadores.

II-A. Fase 1: Representación

En esta primera fase, se evaluará como la representación afecta al algoritmo NEAT. Para esta fase, se evaluarán dos representaciones diferentes; además, se probarán distintos porcentajes de dimensión para las representaciones elegidas y también se probará el tamaño de la población. Los parámetros a evaluar son:

- **Representación**: TF-IDF, Word Embeddings
- **Dimensiones**: 20 %, 25 %, 30 %

Tabla I
CONFIGURACIONES PARA NEAT

Parámetro	Valor
Representación	WE, TFIDF
Dimensiones	20 %, 25 %, 30 %
Población	100, 200, 400
Generaciones	50
Inicialización de conexiones	full_direct
Añadir nodo	0.10, 0.20, 0.30
Añadir conexión	0.05, 0.10, 0.20
Eliminar conexión	0.05, 0.10, 0.20
Compatibilidad	2.5, 3.0, 5.0
Mutación de pesos	0.50, 0.70, 0.90
Umbral de supervivencia	0.20, 0.30, 0.50

- **Población:** 100, 200, 400

Para esta sección se fijaran los demás parámetros de la siguiente manera:

- **Añadir nodo:** 0.10
- **Añadir conexión:** 0.10
- **Eliminar conexión:** 0.10
- **Compatibilidad:** 2.5
- **Mutación de pesos:** 0.50
- **Umbral de supervivencia:** 0.50

Después de realizar la experimentación, se obtuvieron los resultados mostrados en la *Tabla: II*. Dichos resultados nos demuestran un desempeño regular en la columna de aptitud, teniendo valores positivos como 0,5629; sin embargo, cuando evaluamos nuestra red con el conjunto de prueba, podemos observar que nuestra métrica **F1-score** decae hasta 0,4630. Podemos observar que la representación que mejor se comporta es **Word Embeddings**, con una dimensionalidad variada entre 20 % y 30 % lo que significa que una representación más pequeña, como los **WE** funcionan mejor para el algoritmo **NEAT**.

Tabla II
RESULTADOS NEAT FASE 1

Dimensiones	Representación	Aptitud	F1 Test	Nodos	Conexiones	Población
0.30	WE	0.5486	0.4630	3	162	400
0.25	WE	0.5629	0.4341	3	136	100
0.20	WE	0.5863	0.4333	5	111	100
0.30	TFIDF	0.4016	0.4325	5	415	200
0.30	TFIDF	0.3779	0.4257	4	411	100

Este resultado es bajo; sin embargo, el conjunto de datos utilizado contiene muy pocos datos de entrenamiento. Considerando la tarea de clasificación, contar únicamente con cerca de 3,000 datos para entrenamiento se vuelve complicado para nuestras redes neuronales. Sumado a un fuerte desbalance de clases, es más complicado para las redes clasificar de manera correcta. Las clases se encuentran distribuidas de la siguiente manera:

- **Neutral:** 1485
- **Positivo:** 968
- **Negativo:** 689

Es por esta razón que se obtuvo una segunda métrica para comparar el **F1-score** de cada una de las clases, y esta

será la que se evaluará. Para la evaluación, se entreno un modelo **SVM** con la misma representación (**WE**), la misma dimensionalidad (30 %) y el kernel **rbf**; una vez entrenado, este modelo se evaluó con el mismo conjunto de test. A su vez, se comparo con una red neuronal de tres capas (512, 256, 64) entrenada en PyTorch utilizada en [1] para comparar con una red entrenada a mano con el mismo conjunto de datos. La *Tabla: III* muestra los resultados de los modelos.

Tabla III
F1-SCORE POR CLASES FASE 1

Modelo	F1 Test	F1-negative	F1-neutral	F1-positive
NEAT	0.4630	0.3488	0.5986	0.4417
SVM	0.6575	0.5233	0.7755	0.6739
MLP_B	0.6630	0.5607	0.7524	0.6758

Como podemos observar, en general, los modelos **SVM** y **MLP** obtuvieron resultados muy parecidos. Sin embargo, el modelo de **NEAT** sigue muy por debajo de dichos modelos, por lo cual se seguirán realizando distintos experimentos para obtener un mejor resultado y ver que parámetros son los más significativos para el algoritmo, ya que actualmente solo se probó la representación y la población.

II-B. Fase 2: Estructura

Con los resultados obtenidos en la fase anterior en la *Tabla: II* fijaremos los parámetros obtenidos para enfocarnos únicamente en la estructura de nuestras redes. Los parámetros fijados serán:

- **Población:** 400
- **Dimensiones:** 0.30
- **Representación:** WE
- **Compatibilidad:** 2.5
- **Mutación de pesos:** 0.5
- **Umbral de supervivencia:** 0.5

Y para evaluar como afecta la estructura de las redes neuronales al algoritmo **NEAT**. Los valores a probar en esta fase serán:

- **Añadir nodo:** 0.10, 0.20, 0.30
- **Añadir conexión:** 0.05, 0.10, 0.20
- **Eliminar conexión:** 0.05, 0.10, 0.20

Con estas configuraciones nuevamente repetiremos los experimentos y obtendremos las mejores configuraciones. En la *Tabla: IV* se muestran los resultados de los experimentos y la configuración ganadora en esta ocasión es igual a la de la fase 1, pues obtuvieron la misma aptitud y el mismo **F1-score**. Esto nos dice que los valores iniciales seleccionados de manera aleatoria al final fueron los mejores.

Tabla IV
RESULTADOS NEAT FASE 2

Aptitud	F1 Test	Añadir nodo	Añadir conexión	Eliminar conexión
0.5486	0.4630	0.10	0.10	0.10
0.5960	0.4492	0.30	0.05	0.10
0.5301	0.4438	0.20	0.20	0.10
0.5734	0.4363	0.30	0.05	0.20
0.5677	0.4286	0.30	0.20	0.05

Esta fase nos demuestra que los parámetros probados no fueron tan relevantes, ya que, a pesar de los experimentos, no se logró aumentar de manera drástica el **F1** en el conjunto de prueba. Esto se debe principalmente a que la estructura interna de la red propuesta por **NEAT** no es capaz de igualar a las arquitecturas de **SVM** y **MLP_B**; ya que la red devuelta por **NEAT** tiene solamente *nodos_finales* = 4 y *conexiones_finales* = 165, lo que nos sugiere que se trata de una red con topología pequeña; sin embargo, altamente conectada, esto le permite a la red de **NEAT** capturar relaciones no lineales sin necesidad de capas profundas, el problema está en que esta topología reducida no logró igualar a una arquitectura profunda con miles de neuronas. Es por eso que en la siguiente fase se probará como la diversidad beneficia a nuestro algoritmo a encontrar un mejor óptimo local con topologías más grandes o mejor conectadas.

II-C. Fase 3: Diversidad

Con los resultados de los experimentos anteriores, en este ultimo experimento probaremos como la diversidad del algoritmo y el tener más especies compitiendo dentro del algoritmo ayuda a mejorar el **F1-score**. Para esta ultima fase dejaremos fijados los siguientes parámetros:

- **Población:** 400
- **Dimensiones:** 0.30
- **Representación:** WE
- **Añadir nodo:** 0.1,
- **Añadir conexión:** 0.1
- **Eliminar conexión:** 0.1

Probar de esta manera los últimos parámetros propuestos, los cuales son:

- **Compatibilidad:** 2.5, 3.0, 5.0
- **Mutación de pesos:** 0.5, 0.7, 0.9
- **Umbral de supervivencia:** 0.2, 0.3, 0.5

Con estos parámetros se espera experimentar si la diversidad puede generar topologías más complejas que ayuden a captar las relaciones tan complejas del conjunto de datos.

Una vez realizados los experimentos, se obtuvieron los resultados mostrados en la *Tabla: V*. En esta tabla podemos observar que el **F1-score** incrementó de 0,4630 hasta 0,4909. Esto es una señal de que los parámetros de compatibilidad fueron importantes en esta ultima fase, ya que, de manera general, los cinco mejores individuos mejoraron respecto a la fase anterior; sin embargo, aún se encuentra por debajo de los modelos rivales.

Tabla V
RESULTADOS NEAT FASE 3

Fitness	F1 Test	Compatibilidad	Mutación	Supervivencia
0.5759	0.4909	5.0	0.50	0.20
0.5417	0.4717	5.0	0.50	0.50
0.5651	0.4656	3.0	0.70	0.30
0.5885	0.4596	2.5	0.90	0.50
0.6071	0.4533	2.5	0.90	0.20

Para evaluar de manera individual que tanto mejoró el modelo de **NEAT** en la clasificación por clases, se obtuvo nuevamente la métricas por separado, la *Tabla: VI* muestra dicha comparación. A pesar de que el modelo de **NEAT** no logro superar a sus modelos competidores si logró superarse asimismo respecto a las versiones anteriores, por lo que el algoritmo funciona y devuelve individuos competentes; sin embargo, estos no se comparan con una red neuronal multicapa (**MLP**) o un clasificador **SVM** en cuanto capacidad para modelar relaciones complejas, cosa para lo cual los modelos competidores están diseñados.

Tabla VI
F1-SCORE POR CLASES FASE 3

Modelo	F1 Test	F1-negative	F1-neutral	F1-positive
NEAT	0.4909	0.3794	0.6279	0.4654
SVM	0.6575	0.5233	0.7755	0.6739
MLP_B	0.6630	0.5607	0.7524	0.6758

En esta ultima fase pudimos comprobar que la diversidad es un componente importante para la evolución de nuestro algoritmo **NEAT**, siendo uno de los parámetros más importantes dentro de los parámetros probados; sin embargo, **NEAT** cuenta con mucho más parámetros que en investigaciones futuras sería importante realizar más experimentos probando dichos parámetros.

III. CONCLUSIONES

El hecho de que **NEAT** no haya logrado superar el rendimiento de clasificadores tradicionales como **SVM** o arquitecturas profundas fijas (**MLP**) no representa un fallo en la implementación, sino una manifestación de los límites teóricos del algoritmo en dominios de alta dimensionalidad, como la clasificación de texto. Mientras que una **SVM** optimiza en un hiperplano y las redes **MLP** aprovechan el gradiente descendente para dirigir de manera exacta la actualización de pesos, **NEAT** se enfrenta a un desafío doble gigante: optimizar la topología y ajustar los pesos sin guía analítica. Encontrar una combinación exacta mediante mutaciones aleatorias en vectores de entrada de gran tamaño resulta sumamente complejo. La representación mediante **Word Embeddings** demostró un comportamiento superior en comparación con **TF-IDF**. Los vectores densos de **FastText** capturan relaciones semánticas que facilitan la convergencia evolutiva. Asimismo, el análisis de sensibilidad demostró que **NEAT** se beneficia de dimensionalidades compactas (30 %), donde el espacio de

entradas es lo suficientemente rico, pero no tan expansivo como para ralentizar la evolución incremental.

Se observó que las redes resultantes de NEAT tienden a conservar topologías pequeñas, pero con una densidad de conectividad elevada (por ejemplo, redes con 4 nodos y 165 conexiones en la Fase 2). Sin embargo, frente a un severo desbalance de clases, estas estructuras reducidas carecen de la capacidad de memorización y generalización que poseen las MLP con miles de parámetros entrenados.

La Fase 3 evidenció que el manejo de la diversidad es el componente dinámico más potente dentro del archivo de configuración de NEAT. Al elevar el umbral de compatibilidad a 5.0 y restringir el umbral de supervivencia a 0.20, se protegió la innovación topológica en subespecies competitivas, permitiendo un incremento directo del **F1-score** de test de 0.4630 a 0.4909. Esto valida la teoría original de NEAT: preservar la diversidad estructural es indispensable para que el algoritmo escape de óptimos locales planos durante las mutaciones.

En conclusión, NEAT no debe interpretarse como un sustituto directo de los clasificadores tradicionales en escenarios de texto, sino como un laboratorio evolutivo capaz de explorar arquitecturas novedosas y compactas. Su mayor aporte radica en demostrar que la diversidad estructural y la representación semántica adecuada son factores críticos para que los algoritmos evolutivos puedan competir en tareas donde los métodos clásicos aún dominan.

REFERENCIAS

- [1] Brian Miguel Escalona Maldonado. Clasificación multi-clase con redes neuronales feedforward usando pytorch. *UACM*, 2025.

APÉNDICE

```
1 import itertools
2 import multiprocessing
3 import pandas as pd
4 from sklearn.model_selection import
    ↪ train_test_split
5 from sklearn.preprocessing import LabelEncoder
6 from sklearn import svm
7 from sklearn.metrics import f1_score
8 import numpy as np
9 import neat
10 from sklearn.preprocessing import
    ↪ StandardScaler
11 from sklearn.decomposition import PCA
12 from sklearn.model_selection import
    ↪ train_test_split
13 from scipy.special import softmax
14 from nltk.corpus import stopwords
15 from sklearn.feature_extraction.text import
    ↪ TfidfVectorizer, CountVectorizer
16 import unicodedata
17 import re
```

```
18 import configparser
19 from functools import partial
20 from sklearn.decomposition import TruncatedSVD
21 from typing import List, Dict, Any, Tuple
22
23 SEED: int = 42
24 np.random.seed(SEED)
25
26 GLOBAL_X_BATCH: np.ndarray | None = None
27 GLOBAL_Y_BATCH: np.ndarray | None = None
28
29 _STOPWORDS: List[str] =
    ↪ stopwords.words("spanish")
30 PUNCTUATION: str = ";:,.\\-\\\"'/"
31 SYMBOLS: str = "()[]?;!()~<>|@#"
32 NUMBERS: str = "0123456789"
33 SKIP_SYMBOLS: set[str] = set(PUNCTUATION +
    ↪ SYMBOLS)
34 SKIP_SYMBOLS_AND_SPACES: set[str] =
    ↪ set(PUNCTUATION + SYMBOLS + '\t\n\r ')
35
36
37 def normaliza_texto(
38     input_str: str,
39     punct: bool = False,
40     accents: bool = False,
41     num: bool = False,
42     max_dup: int = 2
43 ) -> str:
44     """
45     Normaliza texto eliminando puntuación,
46     ↪ acentos, números y duplicados.
47     """
48     nfkd_f: str =
49     ↪ unicodedata.normalize('NFKD',
50     ↪ input_str)
51     n_str: List[str] = []
52     c_prev: str = ''
53     cc_prev: int = 0
54     for c in nfkd_f:
55         if not num and c in NUMBERS:
56             continue
57         if not punct and c in SKIP_SYMBOLS:
58             continue
59         if not accents and
60         ↪ unicodedata.combining(c):
61             continue
62         if c_prev == c:
63             cc_prev += 1
64             if cc_prev >= max_dup:
65                 continue
66         else:
67             cc_prev = 0
68     n_str.append(c)
69     c_prev = c
```

```

66 texto: str = unicodedata.normalize('NFKD',
    ↪  "".join(n_str))
67 texto = re.sub(r'(\s)+', r' ',
    ↪  texto.strip(), flags=re.IGNORECASE)
68 return texto
69
70
71 def mi_preprocesamiento(texto: str) -> str:
72     """Convierte a minúsculas y normaliza el
    ↪  texto."""
73     return normaliza_texto(texto.lower())
74
75
76 def mi_tokenizador(texto: str) -> List[str]:
77     """Tokeniza eliminando stopwords."""
78     return [t for t in texto.split() if t not
    ↪  in _STOPWORDS]
79
80
81 def update_neat_config(config_path: str,
    ↪  cambios: Dict[str, Any]) -> neat.Config:
82     """Actualiza parámetros de configuración
    ↪  NEAT y devuelve un objeto Config."""
83     conf = configparser.ConfigParser()
84     conf.read(config_path)
85
86     for param, valor in cambios.items():
87         if param == 'pop_size':
88             conf.set('NEAT', param,
    ↪  str(valor))
89         elif param in [
90             'num_inputs',
91             'num_outputs',
92             'node_add_prob',
93             'conn_add_prob',
94             'conn_delete_prob',
95             'weight_mutate_rate'
96         ]:
97             conf.set('DefaultGenome', param,
    ↪  str(valor))
98         elif param ==
    ↪  'compatibility_threshold':
99             conf.set('DefaultSpeciesSet',
    ↪  param, str(valor))
100         elif param == 'survival_threshold':
101             conf.set('DefaultReproduction',
    ↪  param, str(valor))
102
103 temp_config: str = "temp_config.cfg"
104 with open(temp_config, "w") as f:
105     conf.write(f)
106
107 return neat.Config(
108     neat.DefaultGenome,
109     neat.DefaultReproduction,
110     neat.DefaultSpeciesSet,

```

```

111     neat.DefaultStagnation,
112     temp_config
113 )
114
115
116 def eval_genome(
117     genome: neat.DefaultGenome,
118     config: neat.Config,
119     X_batch: np.ndarray,
120     Y_batch: np.ndarray
121 ) -> float:
122     """Evalúa un genoma con NEAT usando F1
    ↪  macro y penalización por
    ↪  complejidad."""
123     net =
    ↪  neat.nn.FeedForwardNetwork.create(genome,
    ↪  config)
124     predictions: np.ndarray =
    ↪  np.empty(len(X_batch), dtype=np.int32)
125
126     for i, xi in enumerate(X_batch):
127         predictions[i] =
    ↪  np.argmax(net.activate(xi))
128
129     fitness: float = f1_score(Y_batch,
    ↪  predictions, average="macro")
130     complexity_penalty: float =
    ↪  len(genome.connections) * 0.0005
131     return fitness - complexity_penalty
132 if __name__ == "__main__":
133     # Carga de datasets
134     dataset_train: pd.DataFrame =
    ↪  pd.read_json(
135
    ↪  "./data/dataset_polaridad_es_train_embedding
    ↪  lines=True
136 )
137     dataset_test: pd.DataFrame = pd.read_json(
    ↪  "./data/dataset_polaridad_es_test_embeddings
    ↪  lines=True
138 )
139
140
141 le: LabelEncoder = LabelEncoder()
142
143 # Definición de rangos de experimentos
144 experimentos: Dict[str, List[Any]] = {
145     'pop_size': [100, 200, 400],
146     'node_add_prob': [0.1],
147     'conn_add_prob': [0.1],
148     'conn_delete_prob': [0.1],
149     'compatibility_threshold': [2.5],
150     'weight_mutate_rate': [0.5],
151     'survival_threshold': [0.5]
152 }
153

```

```

154 # Representaciones a probar
155 representaciones: List[str] = ['TFIDF',
    ↳ 'WE']
156
157 # Porcentajes de reducción dimensional
158 dimensiones: List[float] = [0.20, 0.25,
    ↳ 0.30]
159
160 # Lista para almacenar resultados
161 resultados_totales: List[Dict[str, Any]] =
    ↳ []
162
163 # Todas las combinaciones de parámetros
164 combinaciones: List[Tuple[Any, ...]] =
    ↳ list(itertools.product(*experimentos.valores))
165
166 for repr_type in representaciones:
167     for dim_pct in dimensiones:
168         # Preparación de datos según
169         ↳ representación
170         if repr_type == 'TFIDF':
171             X_train: np.ndarray =
172             ↳ dataset_train["text"].to_numpy()
173             Y_train: np.ndarray =
174             ↳ dataset_train["klass"].to_numpy()
175             X_test: np.ndarray =
176             ↳ dataset_test["text"].to_numpy()
177             Y_test: np.ndarray =
178             ↳ dataset_test["klass"].to_numpy()
179
180             Y_encoded: np.ndarray =
181             ↳ le.fit_transform(Y_train)
182             Y_test_encoded: np.ndarray =
183             ↳ le.transform(Y_test)
184
185             vec_tfidf: CountVectorizer =
186             ↳ CountVectorizer(
187                 analyzer="word",
188                 preprocessor=mi_preprocesador,
189                 tokenizer=mi_tokenizador,
190                 ngram_range=(1, 1),
191                 max_features=3000,
192                 min_df=2,
193                 max_df=0.9
194             )
195             X_tfidf: np.ndarray =
196             ↳ vec_tfidf.fit_transform(X_train)
197             X_test_tfidf: np.ndarray =
198             ↳ vec_tfidf.transform(X_test)
199
200             total_features: int =
201             ↳ X_tfidf.shape[1]
202             n_comp: int =
203             ↳ int(total_features *
204                 dim_pct)

```

```

192 svd: TruncatedSVD =
    ↳ TruncatedSVD(n_components=n_comp,
    ↳ random_state=42)
193
194 X_reduced: np.ndarray =
    ↳ svd.fit_transform(X_tfidf)
195 X_test: np.ndarray =
    ↳ svd.transform(X_test_tfidf)
196
197 X_train, X_val,
    ↳ Y_train_encoded,
    ↳ Y_val_encoded =
    ↳ train_test_split(
        X_reduced,
        Y_encoded,
        test_size=0.2,
        random_state=42,
        stratify=Y_encoded
    )
198
199 else:
200     X_trainext: np.ndarray =
201     ↳ dataset_train['text'].to_numpy()
202     X_es: np.ndarray =
203     ↳ np.vstack(dataset_train["we_es"].to_numpy(),
204                 dataset_train["we_mx"].to_numpy(),
205                 dataset_train["we_ft"].to_numpy())
206     X_mx: np.ndarray =
207     ↳ np.vstack(dataset_train["we_mx"].to_numpy(),
208                 dataset_train["we_ft"].to_numpy(),
209                 dataset_train["we_es"].to_numpy())
210     X_ft: np.ndarray =
211     ↳ np.vstack(dataset_train["we_ft"].to_numpy(),
212                 dataset_train["we_es"].to_numpy(),
213                 dataset_train["we_mx"].to_numpy())
214     Y_train: np.ndarray =
215     ↳ dataset_train['klass'].to_numpy()
216
217     X_train: np.ndarray =
218     ↳ np.concatenate([X_es, X_mx, X_ft], axis=1)
219
220     X_es_t: np.ndarray =
221     ↳ np.vstack(dataset_test["we_es"].to_numpy(),
222                 dataset_test["we_mx"].to_numpy(),
223                 dataset_test["we_ft"].to_numpy())
224     X_mx_t: np.ndarray =
225     ↳ np.vstack(dataset_test["we_mx"].to_numpy(),
226                 dataset_test["we_ft"].to_numpy(),
227                 dataset_test["we_es"].to_numpy())
228     X_ft_t: np.ndarray =
229     ↳ np.vstack(dataset_test["we_ft"].to_numpy(),
230                 dataset_test["we_es"].to_numpy(),
231                 dataset_test["we_mx"].to_numpy())
232     X_test: np.ndarray =
233     ↳ np.concatenate([X_es_t, X_mx_t, X_ft_t], axis=1)
234     Y_test: np.ndarray =
235     ↳ dataset_test['klass'].to_numpy()
236
237     scaler: StandardScaler =
238     ↳ StandardScaler()
239     X_train =
240     ↳ scaler.fit_transform(X_train)
241     X_test =
242     ↳ scaler.transform(X_test)

```

```

224 Y_train_encoded: np.ndarray = 257
    ↳ le.fit_transform(Y_train)
225 Y_test_encoded: np.ndarray = 258
    ↳ le.transform(Y_test)
226
227 total_features: int = 259
    ↳ X_train.shape[1]
228 n_comp: int = 260
    ↳ int(total_features *
    ↳ dim_pct)
229 pca: PCA = 261
    ↳ PCA(n_components=n_comp)
230
231 X_train = 262
    ↳ pca.fit_transform(X_train)
232 X_test = pca.transform(X_test) 263
233
234 X_train, X_val,
    ↳ Y_train_encoded,
    ↳ Y_val_encoded =
    ↳ train_test_split(
235     X_train, 266
236     Y_train_encoded,
237     test_size=0.2,
238     random_state=42, 267
239     stratify=Y_train_encoded
240 )
241
242 input_size: int = X_train.shape[1] 268
243
244 for params in combinaciones: 269
245     dict_cambios: Dict[str, Any] = 270
246     ↳ dict(zip(experimentos.keys(),
247     ↳ params)) 271
248     dict_cambios['num_inputs'] = 272
249     ↳ input_size 273
250
251     print(f"Probando: {repr_type} 274
252     ↳ | Params: {dict_cambios}, 275
253     ↳ Total:
254     ↳ {len(combinaciones)}") 276
255
256     config_actual: neat.Config = 277
257     ↳ update_neat_config('config-5e
258     ↳ dict_cambios)
259
260     p: neat.Population =
261     ↳ neat.Population(config_actual)
262
263     batch_size: int = min(128,
264     ↳ len(X_train))
265     indices: np.ndarray =
266     ↳ np.random.choice(len(X_train),
267     ↳ batch_size, replace=False)
268
269     X_batch: np.ndarray =
270     ↳ X_train[indices]
271     Y_batch: np.ndarray =
272     ↳ Y_train_encoded[indices]
273
274     eval_function =
275     ↳ partial(eval_genome,
276     ↳ X_batch=X_batch,
277     ↳ Y_batch=Y_batch)
278     pe: neat.ParallelEvaluator =
279     ↳ neat.ParallelEvaluator(4,
280     ↳ eval_function)
281
282     winner: neat.DefaultGenome =
283     ↳ p.run(pe.evaluate, 50)
284
285     winner_net:
286     ↳ neat.nn.FeedForwardNetwork
287     ↳ =
288     ↳ neat.nn.FeedForwardNetwork.create(wi
289     ↳ config_actual)
290     test_preds: List[int] =
291     ↳ [np.argmax(winner_net.activate(xi))
292     ↳ for xi in X_test]
293     fl_test: float =
294     ↳ fl_score(Y_test_encoded,
295     ↳ test_preds,
296     ↳ average="macro")
297
298     resultados_totales.append({
299     'Dimensiones': dim_pct,
300     'Representacion':
301     ↳ repr_type,
302     'Fitness_Evolutivo':
303     ↳ winner.fitness,
304     'Fl_Test': fl_test,
305     'Nodos_Finales':
306     ↳ len(winner.nodes),
307     'Conexiones_Finales':
308     ↳ len(winner.connections),
309     **dict_cambios
310     })
311
312     df: pd.DataFrame =
313     ↳ pd.DataFrame(resultados_totales)
314     df.to_csv("reporte_experimentos_neat.csv",
315     ↳ index=False)

```